COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	09/08/2026	**DATE SUBMITTED:**	09/08/2026
NAME:	KEITH NICOLE SILVOZA	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

ACTIVITY NO. 7.1 SORTING ALGORTHMS PT1

1. PROCEDURE OUTPUT

What procedures did you perform?

6. Output

Code + Console Screenshot	
Observations	

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observations	

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	
Observations	

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	
Observations	

Table 7-4. Insertion Sort Algorithm

7. Supplementary Activity

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:

- Pseudocode of Algorithm
- Screenshot of Algorithm Code
- Output Testing

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Table 7-1. Array of Values for Sort Algorithm Testing

CODE	CONSOLE	OBSERVATION

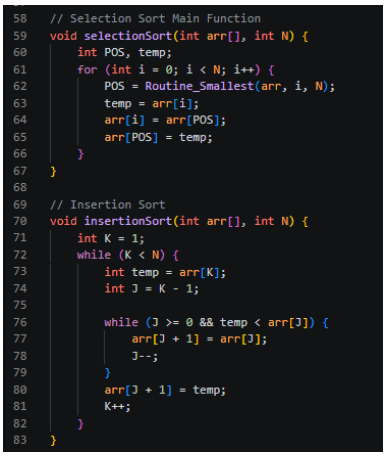
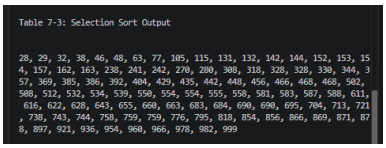
main.cpp
HOA_7.1 > main.cpp > main()
`#include <iostream>`
`#include <cstdlib>`
`#include <ctime>`
 	
`using namespace std;`
 	
`const int SIZE = 100;`
 	
`void generateRandomArray(int arr[], int size) {`
`  for (int i = 0; i < size; i++) {`
`  arr[i] = rand() % 1000 + 1;`
`  }`
`}`
 	
`void printArray(int arr[], int size) {`
`  for (int i = 0; i < size; i++) {`
`  cout << arr[i];`
`  if (i < size - 1) {`
`  cout << ", ";`
`  }`
`  }`
`  cout << endl;`
`}`
 	
`void copyArray(int src[], int dest[], int size) {`
`  for (int i = 0; i < size; i++) {`
`  dest[i] = src[i];`
`  }`
`}
	Table 7-1: Original Unsorted Array (100 elements)
	
	683, 978, 330, 466, 704, 695, 154, 966, 142, 328, 743, 157, 738, 508, 539, 308, 442, 744, 660, 242, 28, 38, 502, 468, 587, 663, 759, 588, 690, 611, 318, 429, 558, 512, 758, 555, 392, 554, 144, 344, 456, 866, 655, 921, 936, 162, 776, 131, 328, 616, 897, 448, 357, 385, 153, 550, 32, 856, 721, 386, 628, 759, 241, 360, 960, 869, 982, 999, 622, 280, 795, 163, 554, 581, 77, 643, 871, 534, 132, 532, 63, 435, 878, 29, 954, 604, 854, 468, 713, 270, 238, 105, 818, 404, 48, 690, 583, 152, 115, 46
		100 random numbers were generated between 1 and 1000. Since the numbers are completely out of order, this array serves as the starting baseline to test all three sorting algorithms.

Table 7-2. Bubble Sort Technique

CODE	CONSOLE	OBSERVATION

`// Bubble Sort`
`void bubbleSort(int arr[], int arrSize) {`
`  for (int i = 0; i < arrSize; i++) {`
`  for (int j = i + 1; j < arrSize; j++) {`
`  if (arr[j] < arr[i]) {`
`  int temp = arr[i];`
`  arr[i] = arr[j];`
`  arr[j] = temp;`
`  }`
`  }`
`  }`
`}`
 	
`// Routine Smallest for Selection Sort`
`int Routine_Smallest(int A[], int K, int arrSize) {`
`  int position = K;`
`  int smallestElem = A[K];`
 	
`  for (int J = K + 1; J < arrSize; J++) {`
`  if (A[J] < smallestElem) {`
`  smallestElem = A[J];`
`  position = J;`
`  }`
`  }`
`  return position;`
`}
	Table 7-2: Bubble Sort Output
	
	28, 29, 32, 38, 46, 48, 63, 77, 105, 115, 131, 132, 142, 144, 152, 153, 154, 157, 162, 163, 238, 241, 242, 270, 280, 308, 318, 328, 328, 330, 344, 357, 369, 385, 386, 392, 404, 429, 435, 442, 448, 456, 466, 468, 468, 502, 508, 512, 532, 534, 539, 550, 554, 554, 555, 558, 581, 583, 587, 588, 611, 616, 622, 628, 643, 655, 660, 663, 683, 684, 690, 690, 695, 704, 713, 721, 738, 743, 744, 758, 759, 759, 776, 795, 818, 854, 856, 866, 869, 871, 878, 897, 921, 936, 954, 960, 966, 978, 982, 999
		The output confirms that the numbers were sorted in ascending order. It works by swapping adjacent elements when the left value is bigger, though it takes a lot of comparisons to push the larger values to the end.

Table 7-3. Selection Sort Algorithm

CODE	CONSOLE	OBSERVATION
		Using `Routine_Smallest`, the algorithm scans for the minimum element first before swapping it into place. The output shows the array is fully sorted, and it does fewer overall swaps than bubble sort.

```
// Selection Sort Main Function
void selectionSort(int arr[], int N) {
    int POS, temp;
    for (int i = 0; i < N; i++) {
        POS = Routine_Smallest(arr, i, N);
        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
    }
}

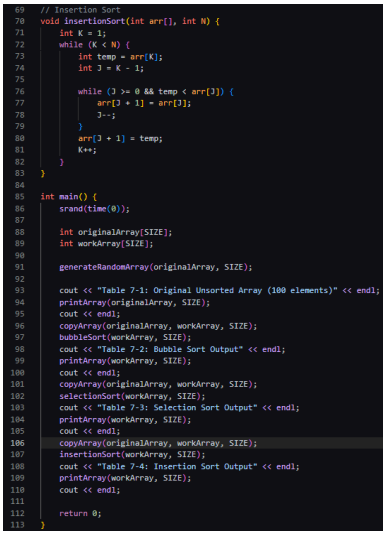
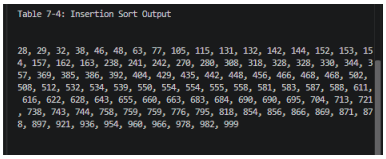
// Insertion Sort
void insertionSort(int arr[], int N) {
    int K = 1;
    while (K < N) {
        int temp = arr[K];
        int J = K - 1;

        while (J >= 0 && temp < arr[J]) {
            arr[J + 1] = arr[J];
            J--;
        }
        arr[J + 1] = temp;
        K++;
    }
}
```

```
Table 7-3: Selection Sort Output

28, 29, 32, 38, 46, 48, 63, 77, 105, 115, 131, 132, 142, 144, 152, 153, 15
4, 157, 162, 163, 238, 241, 242, 270, 280, 308, 318, 328, 328, 330, 344, 3
57, 369, 385, 386, 392, 404, 429, 435, 442, 448, 456, 466, 468, 468, 502,
508, 512, 532, 534, 539, 550, 554, 554, 555, 558, 581, 583, 587, 588, 611,
 616, 622, 628, 643, 655, 660, 663, 683, 684, 690, 690, 695, 704, 713, 721
, 738, 743, 744, 758, 759, 759, 776, 795, 818, 854, 856, 866, 869, 871, 87
8, 897, 921, 936, 954, 960, 966, 978, 982, 999
```

Table 7-4. Insertion Sort Algorithm

CODE	CONSOLE	OBSERVATION
		The output matches the previous sorted results. The code shifts elements to the right one by one to insert `temp` into its correct position, making it a very straightforward process.

```
// Insertion Sort
void insertionSort(int arr[], int N) {
    int K = 1;
    while (K < N) {
        int temp = arr[K];
        int J = K - 1;

        while (J >= 0 && temp < arr[J]) {
            arr[J + 1] = arr[J];
            J--;
        }
        arr[J + 1] = temp;
        K++;
    }
}

int main() {
    srand(time(0));

    int originalArray[SIZE];
    int workArray[SIZE];

    generateRandomArray(originalArray, SIZE);

    cout << "Table 7-1: Original Unsorted Array (100 elements)" << endl;
    printArray(originalArray, SIZE);
    cout << endl;
    copyArray(originalArray, workArray, SIZE);
    bubbleSort(workArray, SIZE);
    cout << "Table 7-2: Bubble Sort Output" << endl;
    printArray(workArray, SIZE);
    cout << endl;
    copyArray(originalArray, workArray, SIZE);
    selectionSort(workArray, SIZE);
    cout << "Table 7-3: Selection Sort Output" << endl;
    printArray(workArray, SIZE);
    cout << endl;
    copyArray(originalArray, workArray, SIZE);
    insertionSort(workArray, SIZE);
    cout << "Table 7-4: Insertion Sort Output" << endl;
    printArray(workArray, SIZE);
    cout << endl;

    return 0;
}
```

```
Table 7-4: Insertion Sort Output

28, 29, 32, 38, 46, 48, 63, 77, 105, 115, 131, 132, 142, 144, 152, 153, 15
4, 157, 162, 163, 238, 241, 242, 270, 280, 308, 318, 328, 328, 330, 344, 3
57, 369, 385, 386, 392, 404, 429, 435, 442, 448, 456, 466, 468, 468, 502,
508, 512, 532, 534, 539, 550, 554, 554, 555, 558, 581, 583, 587, 588, 611,
 616, 622, 628, 643, 655, 660, 663, 683, 684, 690, 690, 695, 704, 713, 721
, 738, 743, 744, 758, 759, 759, 776, 795, 818, 854, 856, 866, 869, 871, 87
8, 897, 921, 936, 954, 960, 966, 978, 982, 999
```

SUPPLEMENTARY ACTIVITY

PSEUDOCODE

```
Start

   Create array votes of size 100
   Create array count of size 6 and set all elements to 0

   For i = 0 to 99:
      votes[i] = random number from 1 to 5
   End For

   For i = 0 to 99:
      pos = i
      smallest = votes[i]

      For j = i + 1 to 99:
         If votes[j] < smallest then:
            smallest = votes[j]
            pos = j
         End If
      End For

      temp = votes[i]
      votes[i] = votes[pos]
      votes[pos] = temp
   End For

   For i = 0 to 99:
      candidateNum = votes[i]
      count[candidateNum] = count[candidateNum] + 1
   End For

   maxVotes = count[1]
   winner = 1

   For c = 2 to 5:
      If count[c] > maxVotes then:
         maxVotes = count[c]
         winner = c
      End If
   End For

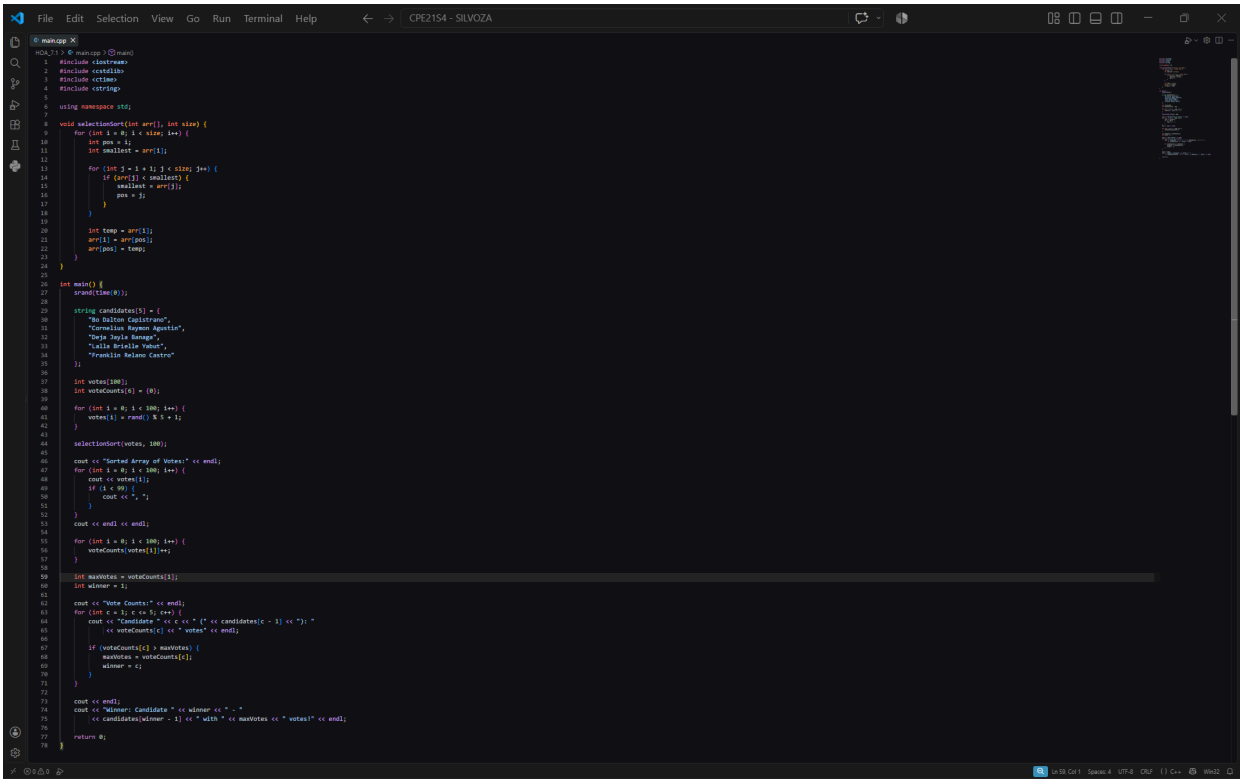
   Print sorted votes array
   Print vote tally for Candidates 1 to 5
   Print winning candidate and vote total

End
```

COMPUTER ENGINEERING

TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

QUEZON CITY



Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
Sorted Array of Votes: 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2

CONCLUSION:

Summary of Lessons Learned I learned how bubble, selection, and insertion sort actually work using 100 random numbers in C++. Using helper functions like Routine Smallest made it easier to understand how elements swap and compare. Sorting the array first also made the vote counting part way simpler to code.

Analysis of the Procedure The lab procedure was clear and easy to follow. Running all three algorithms on the exact same dataset showed how different they are. Bubble sort took too many swaps and felt really repetitive. Selection sort was better since it finds the smallest number first before making any swaps. Insertion sort was the smoothest to use because it just shifts numbers over into place.

Analysis of the Supplementary Activity Selection sort worked great for counting the votes. Sorting the 100 random votes grouped all the 1s, 2s, 3s, 4s, and 5s together right away. That made it super easy to loop through, tally up the totals for each candidate, and see who won without messing up the counts.

Concluding Statement / Feedback I think I did good setting up the C++ functions in VS Code and getting the right terminal outputs. I just need to practice tracing array indices manually on paper so I can debug nested loops faster next time.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.